

Midterm_Berruti

May 6, 2026

1 Motion Analysis Mid-term Assignment

1.1 1. Objective

In this assignment, you will work on **change detection** and **blob matching** tasks.

1.2 2. Methodology Sketch

Change detection is based on a loop where, at each time frame, you compare the current frame I_t with a reference image or background B .

1.2.1 A. Background Initialization

The background is usually initialized (B_0) by taking an average of the first few frames (assuming they are relatively empty/static).

1.2.2 B. Change Detection

At each time t : 1. **Update background B_t** : This is optional. Initially, you may keep it constant ($B_t = B_0$). 2. **Compute change detection map M_t** : Compare B_t with the current frame I_t .

1.2.3 C. Compute Connected Components (Blobs)

- Each group of connected pixels marked as changes is labeled with a common ID.
- *Suggested Libraries*: Use `skimage.measure.label` or OpenCV's `cv2.connectedComponents`.

1.2.4 D. Map Cleaning

- Remove small connected components (noise).
- Apply **morphological operations** (erosion, dilation, opening, closing) to make components smoother, more compact, or to bridge small gaps between components.

1.2.5 E. Blob Matching

Connect components across temporal frames to track motion: - Compute matches between blobs in M_t and M_{t+1} . - **Criteria for matching**: - Vicinity (e.g., Euclidean distance between centroids). - Similarity (e.g., average intensity/color, area, shape descriptors).

1.3 3. Implementation Details

1.3.1 Thresholding

- Start with **heuristics-based thresholds** (\$ au\$) as discussed in class.
- Experiment with **Otsu's Method**: Evaluate if it can be applied to this specific context and provide feedback on its effectiveness.

1.3.2 Matching Strategies

- Experiment with different strategies (Distance-based vs. Descriptor-based).
-

1.4 4. Project Structure

Your code must be **modular, clear, and clean**. Each step (Initialization, Detection, Cleaning, Matching) should be implemented in its own function.

1.5 5. Experimental Analysis

This is a critical part of the assignment. 1. **Provided Video**: Perform your initial analysis and tuning on the provided sample video. 2. **Custom Video**: Acquire a video of your choice. Think about the specific challenges (e.g., lighting changes, occlusion, fast motion) you want to address. 3. **Discussion**: The notebook must include markdown cells reporting, commenting, and analyzing the results.

1.6 6. Optional Extension (NOT DONE)

- Integrate matching results over larger time spans (multi-frame tracking/trajectories).
-

2 Functions

```
[1]: import numpy as np
import matplotlib.pyplot as plt
plt.rcParams["figure.dpi"] = 50
from pathlib import Path
from skimage import measure, morphology, io

#
def initialize_background(video_path="video", num_frames=10):
    video_path = Path(video_path)
    frame_paths = sorted(video_path.glob("*.jpg"))

    if len(frame_paths) < num_frames:
```

```

        raise ValueError(f"Expected at least {num_frames} frames in_
↪{video_path}, found {len(frame_paths)}")

frames = []
for frame_path in frame_paths[:num_frames]:
    frame = io.imread(str(frame_path))
    if frame is None:
        raise FileNotFoundError(f"Could not read frame: {frame_path}")
    if frame.ndim == 3:
        frame = frame.mean(axis=2)    # black and white images
    frames.append(frame.astype(np.float32))

return np.mean(frames, axis=0)

```

```

[2]: #
def load_frames(video_path):
    video_path = Path(video_path)
    frame_paths = sorted(video_path.glob("*.jpg"))
    if not frame_paths:
        raise ValueError(f"No frames found in {video_path}")

    frames = []
    for frame_path in frame_paths:
        frame = io.imread(str(frame_path))
        if frame.ndim == 3:
            frame = frame.mean(axis=2)    # black and white images
        frames.append(frame.astype(np.float32))

    return frames, frame_paths

```

```

[3]: def compute_change_map(frame, background, threshold):
    if frame.shape != background.shape:
        raise ValueError("Frame and background must have the same shape")
    if threshold < 0:
        raise ValueError("Threshold must be non-negative")

    frame = frame.astype(np.float32)
    background = background.astype(np.float32)
    difference = np.abs(frame - background)
    return (difference > threshold).astype(np.uint8) * 255

```

3 V1 - First version to update background

```
[4]: def update_background(background, new_frame, alpha=0.1):
    if background.shape != new_frame.shape:
        raise ValueError("Background and new frame must have the same shape")
    if alpha < 0 or alpha > 1:
        raise ValueError("Alpha must be between 0 and 1")
    return alpha * new_frame + (1 - alpha) * background

def compute_change_maps_with_update_background(frames, initial_background,
    ↪threshold, alpha=0.1):
    background = initial_background.astype(np.float32)
    change_maps = []

    for frame in frames:
        change_map = compute_change_map(frame, background, threshold)
        change_maps.append(change_map)
        background = update_background(background, frame, alpha=alpha)

    return change_maps
```

4 V2 (v3 in slides) - Second version to update background

```
[5]: def update_backgroundv2(background, new_frame, previous_frame, alpha=0.1,
    ↪threshold=0.1):
    if background.shape != new_frame.shape:
        raise ValueError("Background and new frame must have the same shape")
    if alpha < 0 or alpha > 1:
        raise ValueError("Alpha must be between 0 and 1")
    if threshold < 0 or threshold > 1:
        raise ValueError("Threshold must be between 0 and 1")
    if np.abs(new_frame - previous_frame).mean() > threshold:
        return background
    return alpha * new_frame + (1 - alpha) * background

def compute_change_maps_with_update_backgroundv2(frames, initial_background,
    ↪threshold, alpha=0.1, motion_threshold=0.1):
    background = initial_background.astype(np.float32)
    change_maps = []

    for index, frame in enumerate(frames):
        change_map = compute_change_map(frame, background, threshold)
        change_maps.append(change_map)

        previous_index = max(0, index - 3)
```

```

        previous_frame = frames[previous_index]
        background = update_backgroundv2(
            background,
            frame,
            previous_frame,
            alpha=alpha,
            threshold=motion_threshold,
        )

    return change_maps

```

```

[6]: def clean_map(m_t):
    passmask = morphology.disk(2)
    return morphology.binary_opening(m_t, passmask)

def compute_blobs(change_map):
    labeled_blobs = measure.label(change_map, connectivity=2)
    cleaned_blobs = morphology.remove_small_objects(labeled_blobs, min_size=20)
    return cleaned_blobs

```

5 Implementation

```

[7]: # Load frames and initialise background for the station video
from pathlib import Path

video_dir = Path("video/station")
frames, frame_paths = load_frames(video_dir)
initial_background = initialize_background(video_path=video_dir, num_frames=5)

```

```

[8]: fig, ax = plt.subplots(figsize=(8, 5))
ax.imshow(initial_background, cmap="gray",
          vmin=initial_background.min(), vmax=initial_background.max())
ax.set_title("Initial background B0")
ax.axis("off")
plt.tight_layout()
plt.show()

```

Initial background B0



```
[9]: change_maps_update_background = compute_change_maps_with_update_background(  
    frames,  
    initial_background,  
    threshold=30,  
    alpha=0.1,  
)
```

```
[10]: change_maps_update_backgroundv2 = compute_change_maps_with_update_backgroundv2(  
    frames,  
    initial_background,  
    threshold=30,  
    alpha=0.1,  
    motion_threshold=0.2,  
)
```

```
[11]: print(f"Frames loaded: {len(frames)}")  
    print(f"Change maps with update_background: ␣  
    ↪ {len(change_maps_update_background)}")
```

```

print(f"Change maps with update_backgroundv2:␣
↪{len(change_maps_update_backgroundv2)}")

indices = [0, len(frames) // 2, len(frames) - 1]
fig, axes = plt.subplots(3, 3, figsize=(15, 12))
for row, idx in enumerate(indices):
    axes[row, 0].imshow(frames[idx], cmap="gray")
    axes[row, 0].set_title(f"Frame {idx}")
    axes[row, 0].axis("off")

    axes[row, 1].imshow(change_maps_update_background[idx], cmap="gray",␣
↪vmin=0, vmax=255)
    axes[row, 1].set_title(f"update_background, frame {idx}")
    axes[row, 1].axis("off")

    axes[row, 2].imshow(change_maps_update_backgroundv2[idx], cmap="gray",␣
↪vmin=0, vmax=255)
    axes[row, 2].set_title(f"update_backgroundv2, frame {idx}")
    axes[row, 2].axis("off")

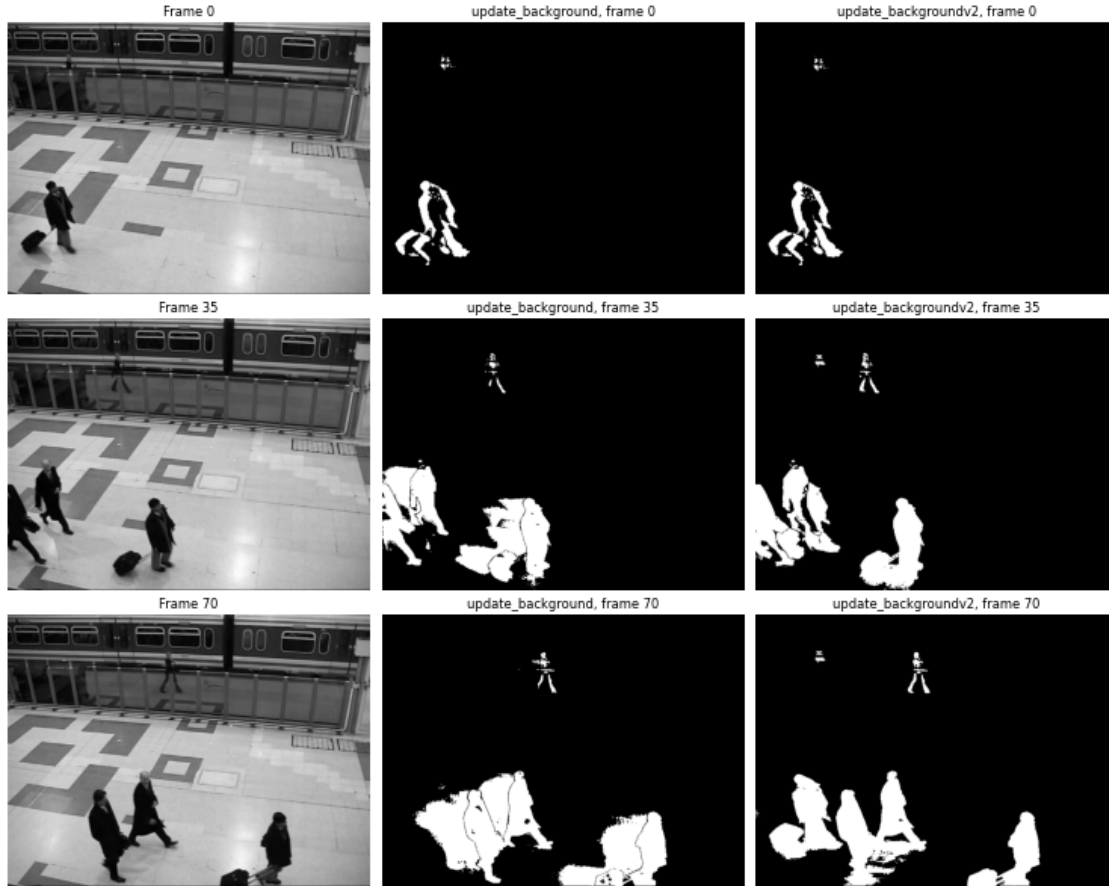
plt.tight_layout()
plt.show()

```

Frames loaded: 71

Change maps with update_background: 71

Change maps with update_backgroundv2: 71



5.1 Analysis: V1 vs V2

V1 (fixed background) — simple and stable. The background never updates, so any slow illumination drift or scene change produces growing false positives over time. Best suited for short sequences under stationary conditions.

V2 (running average) — more robust to slow background changes. The parameter **alpha** controls how fast the background adapts: a small **alpha** (e.g. 0.01) keeps the background stable while fast objects pass through; a larger **alpha** absorbs changes more quickly. The condition on the previous frame ($I_{t-\Delta t}$) corresponds to **Version 3** from the lecture slides: it freezes the background only when the scene has changed with respect to a few frames back, preventing slowly moving objects from being absorbed into the background.

6 Otsu in Action

```
[12]: from skimage.filters import threshold_otsu

def compute_change_map_otsu(frame, background):
    """Compute change map using Otsu's threshold on the difference image.
    Returns the binary map and the computed threshold tau.
    """
    frame = frame.astype(np.float32)
    background = background.astype(np.float32)
    diff = np.abs(frame - background)
    tau = threshold_otsu(diff)
    return (diff > tau).astype(np.uint8) * 255, tau

[13]: def compute_change_maps_otsu(frames, initial_background):
    background = initial_background.astype(np.float32)
    change_maps = []
    thresholds = []
    for frame in frames:
        change_map, tau = compute_change_map_otsu(frame, background)
        change_maps.append(change_map)
        thresholds.append(tau)
    return change_maps, thresholds

[14]: change_maps_otsu, otsu_thresholds = compute_change_maps_otsu(frames,
    ↪initial_background)

print("Otsu threshold - min: {:.1f} max: {:.1f} mean: {:.1f}".format(
    min(otsu_thresholds), max(otsu_thresholds), sum(otsu_thresholds)/
    ↪len(otsu_thresholds)))

Otsu threshold - min: 19.9 max: 74.1 mean: 61.2

[15]: # Visual comparison: fixed threshold vs Otsu on the same frames
indices = [0, len(frames) // 2, len(frames) - 1]
fig, axes = plt.subplots(3, 3, figsize=(15, 12))
for row, idx in enumerate(indices):
    axes[row, 0].imshow(frames[idx], cmap="gray")
    axes[row, 0].set_title("Frame {}".format(idx))
    axes[row, 0].axis("off")

    axes[row, 1].imshow(change_maps_update_background[idx], cmap="gray",
    ↪vmin=0, vmax=255)
    axes[row, 1].set_title("Fixed threshold tau=30 - frame {}".format(idx))
    axes[row, 1].axis("off")

    axes[row, 2].imshow(change_maps_otsu[idx], cmap="gray", vmin=0, vmax=255)
```

```

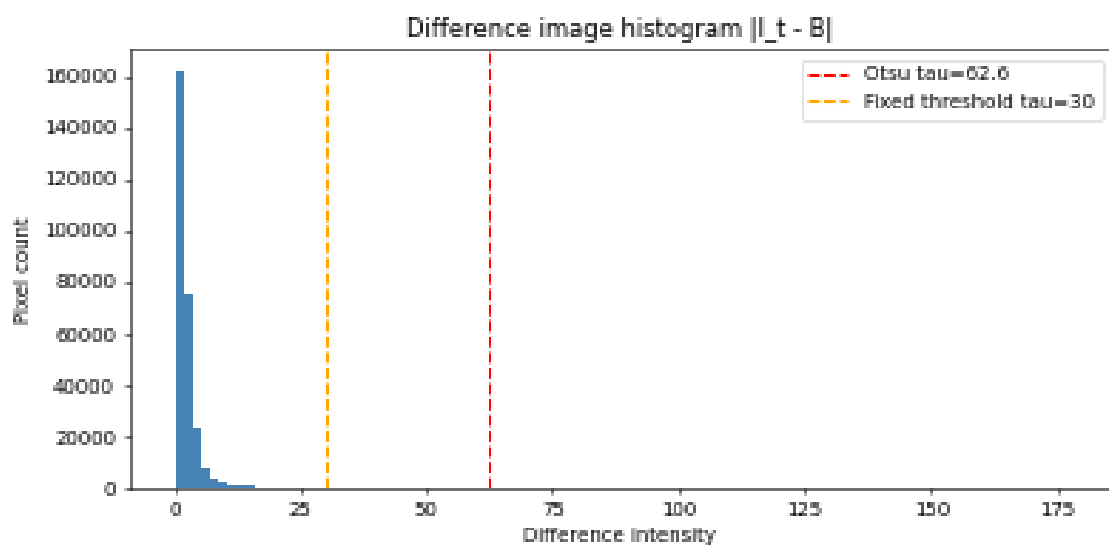
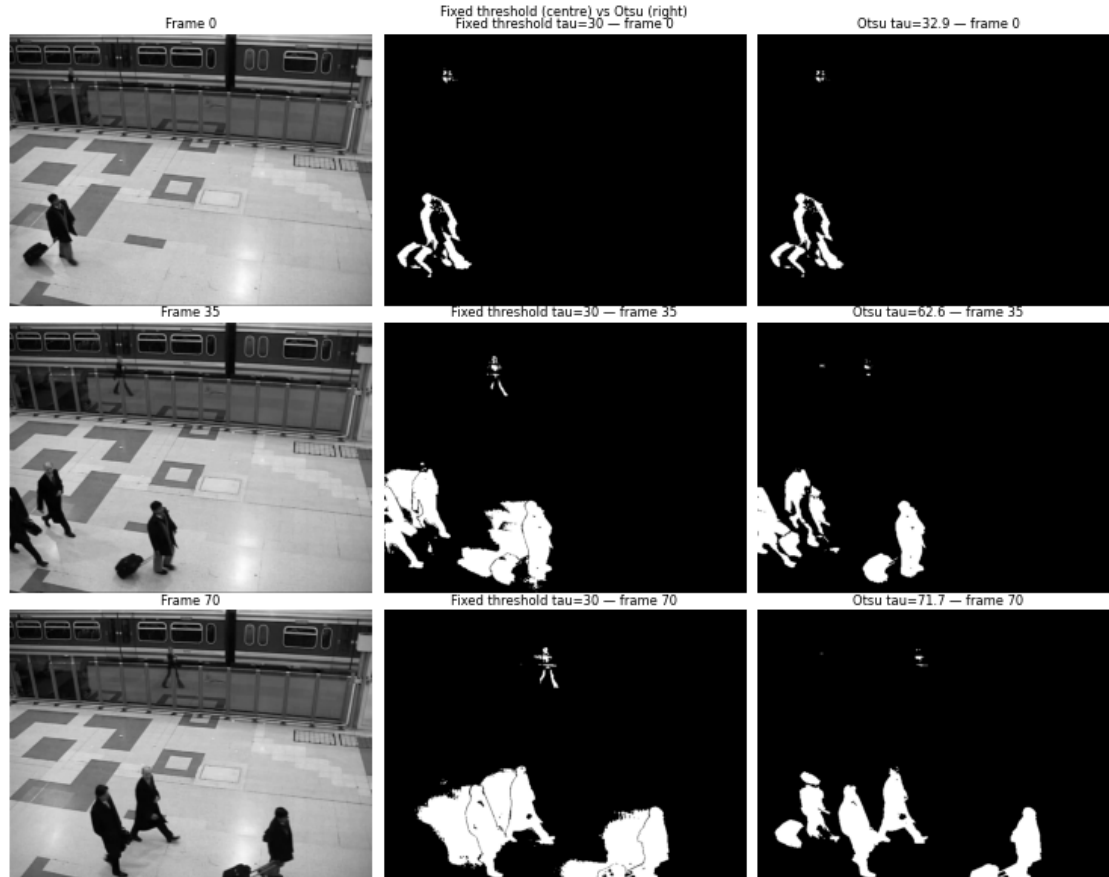
        axes[row, 2].set_title("Otsu tau={:.1f} - frame {}".
        ↪format(otsu_thresholds[idx], idx))
        axes[row, 2].axis("off")

plt.suptitle("Fixed threshold (centre) vs Otsu (right)")
plt.tight_layout()
plt.show()

# Difference histogram for a sample frame
frame_sample = frames[len(frames) // 2].astype(np.float32)
diff_sample = np.abs(frame_sample - initial_background)
tau_otsu_sample = otsu_thresholds[len(frames) // 2]

fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(diff_sample.ravel(), bins=100, color="steelblue", edgecolor="none")
ax.axvline(tau_otsu_sample, color="red", linestyle="--", label="Otsu tau={:.
    ↪1f}".format(tau_otsu_sample))
ax.axvline(30, color="orange", linestyle="--", label="Fixed_
    ↪threshold tau=30")
ax.set_xlabel("Difference intensity")
ax.set_ylabel("Pixel count")
ax.set_title("Difference image histogram |I_t - B|")
ax.legend()
plt.tight_layout()
plt.show()

```



6.1 Analysis: fixed threshold vs Otsu

Otsu outperforms the fixed threshold on this video.

The station background is **not stationary**: artificial lighting and floor reflections change over time, causing the difference image $|I_t - B|$ to grow even in static regions. The fixed threshold $\tau = 30$, calibrated on the first frames, becomes too low for later frames and produces large false-positive blobs on the floor (frames 35 and 70).

Otsu adapts by raising τ automatically ($33 \rightarrow 62 \rightarrow 72$), effectively tracking the drift in background intensity and suppressing these illumination artefacts.

When Otsu works well: when the background drifts slowly over time and the proportion of moving pixels is large enough to create a visible shoulder in the histogram.

When Otsu fails: on frames with very few moving pixels (e.g. an almost empty scene), the histogram is strongly unimodal and Otsu places the threshold inside the noise tail, yielding an unstable or overly low threshold.

7 Blobs on station

```
[16]: def match_blobs_distance(blobs_t, blobs_t_plus_1, max_dist=80):  
    """Distance-based: abbina ogni blob in t al blob piu vicino in t+1  
    usando la distanza euclidea tra centroidi.  
    Returns: lista di tuple (label_t, label_t1, distance)  
    """  
    props_t = measure.regionprops(blobs_t) # calcola le proprietà dei blob in t  
    props_t1 = measure.regionprops(blobs_t_plus_1)  
    if not props_t or not props_t1:  
        return []  
  
    matches = []  
    for p in props_t:  
        cy, cx = p.centroid  
        best_label, best_dist = None, float('inf')  
        for q in props_t1:  
            qy, qx = q.centroid  
            d = np.hypot(cy - qy, cx - qx) # distanza euclidea  
            if d < best_dist:  
                best_dist, best_label = d, q.label  
        if best_dist < max_dist: #  
            matches.append((p.label, best_label, best_dist))  
    return matches
```

```
[17]: def match_blobs_descriptor(blobs_t, blobs_t_plus_1, frame_t, frame_t1,  
                                max_dist=80, w_pos=1.0, w_area=0.5, w_intensity=0.  
                                ↪3):
```

```

"""Descriptor-based: combina distanza tra centroidi con similarita
di area e intensita media del blob nell'immagine originale.

score = w_pos * dist_centroid
      + w_area * |area_t - area_t1| / max(area_t, area_t1)
      + w_intensity * |mean_int_t - mean_int_t1| / 255

Returns: lista di tuple (label_t, label_t1, score)
"""

props_t = measure.regionprops(blobs_t, intensity_image=frame_t.astype(np.
↪float32))
props_t1 = measure.regionprops(blobs_t_plus_1, intensity_image=frame_t1.
↪astype(np.float32))
if not props_t or not props_t1:
    return []
matches = []
for p in props_t:
    cy, cx = p.centroid
    best_label, best_score = None, float('inf')
    for q in props_t1:
        qy, qx = q.centroid
        dist = np.hypot(cy - qy, cx - qx)
        if dist >= max_dist:
            continue
        area_sim = abs(p.area - q.area) / max(p.area, q.area)
        int_sim = abs(p.mean_intensity - q.mean_intensity) / 255.0
        score = w_pos * dist + w_area * area_sim + w_intensity * int_sim
        if score < best_score:
            best_score, best_label = score, q.label
    if best_label is not None:
        matches.append((p.label, best_label, best_score))
return matches

```

```

[18]: def visualize_matches(frame_t, frame_t1, blobs_t, blobs_t1, matches,
↪title='Blob Matching'):
    """Visualizza i blob nelle due mappe affiancate con centroidi colorati per
↪match."""
    fig, axes = plt.subplots(1, 2, figsize=(14, 6))
    axes[0].imshow(frame_t, cmap='gray')
    axes[0].set_title('Frame t  ({} blobs)'.format(blobs_t.max()))
    axes[0].axis('off')
    axes[1].imshow(frame_t1, cmap='gray')
    axes[1].set_title('Frame t+1  ({} blobs)'.format(blobs_t1.max()))
    axes[1].axis('off')
    props_t = {p.label: p for p in measure.regionprops(blobs_t)}
    props_t1 = {p.label: p for p in measure.regionprops(blobs_t1)}
    colors = plt.cm.tab10.colors

```

```

for i, (l_t, l_t1, score) in enumerate(matches):
    color = colors[i % len(colors)]
    if l_t in props_t:
        cy, cx = props_t[l_t].centroid
        axes[0].plot(cx, cy, 'o', color=color, markersize=8)
        axes[0].annotate(str(l_t), (cx, cy), color=color, fontsize=8)
    if l_t1 in props_t1:
        qy, qx = props_t1[l_t1].centroid
        axes[1].plot(qx, qy, 'o', color=color, markersize=8)
        axes[1].annotate(str(l_t1), (qx, qy), color=color, fontsize=8)
plt.suptitle(title)
plt.tight_layout()
plt.show()

```

```

[19]: # Demo: compare the two matching strategies on consecutive frames
from pathlib import Path

video_dir = Path('video/station')
frames, frame_paths = load_frames(video_dir)
initial_background = initialize_background(video_path=video_dir, num_frames=10)

change_maps = compute_change_maps_with_update_backgroundv2(
    frames, initial_background, threshold=30, alpha=0.1, motion_threshold=0.01
)

t = len(frames) // 2 # pick the central frame

change_map_t = clean_map(change_maps[t])
change_map_t1 = clean_map(change_maps[t + 1])

blobs_t = compute_blobs(change_map_t)
blobs_t1 = compute_blobs(change_map_t1)

print('Blob_t shape: {}'.format(blobs_t.shape)) # shape of one image
print('Frame t={}: {} blobs'.format(t, blobs_t.max()))
print('Frame t+1={}: {} blobs'.format(t + 1, blobs_t1.max()))

```

```

Blob_t shape: (480, 640)
Frame t=35: 8 blobs
Frame t+1=36: 8 blobs

```

```

[20]: # --- Strategy 1: Distance-based ---
matches_dist = match_blobs_distance(blobs_t, blobs_t1, max_dist=80)
print('\nDistance-based matches: {}'.format(len(matches_dist)))
for l_t, l_t1, d in matches_dist:
    print(' blob {} -> blob {} (dist={:.1f}px)'.format(l_t, l_t1, d))

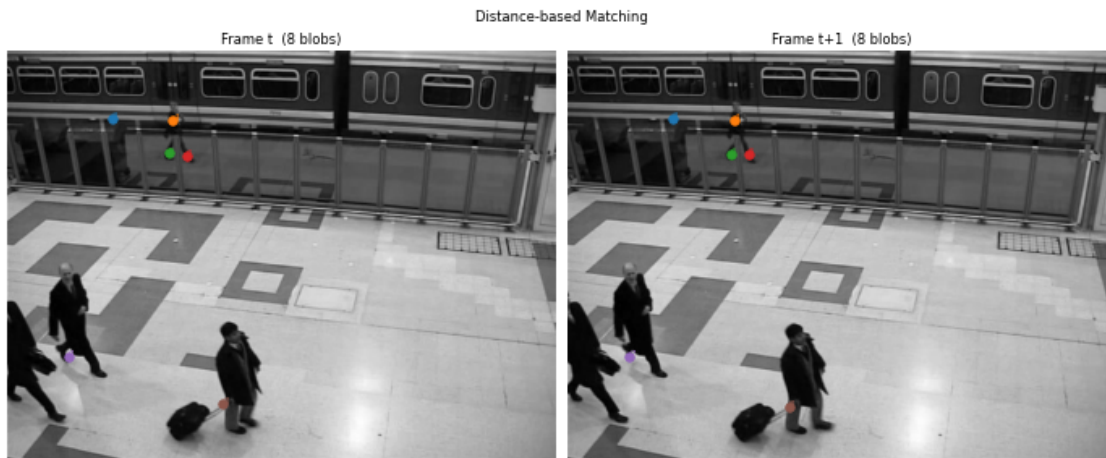
visualize_matches(frames[t], frames[t + 1], blobs_t, blobs_t1,

```

```
matches_dist, title='Distance-based Matching')
```

Distance-based matches: 6

```
blob 3 -> blob 3 (dist=0.0px)
blob 4 -> blob 4 (dist=0.9px)
blob 5 -> blob 6 (dist=1.3px)
blob 6 -> blob 5 (dist=2.1px)
blob 7 -> blob 7 (dist=0.8px)
blob 8 -> blob 8 (dist=7.0px)
```



```
[21]: # --- Strategy 2: Descriptor-based ---
matches_desc = match_blobs_descriptor(blobs_t, blobs_t1,
                                      frames[t], frames[t + 1],
                                      max_dist=80)

print('\nDescriptor-based matches: {}'.format(len(matches_desc)))
for l_t, l_t1, score in matches_desc:
    print(' blob {} -> blob {} (score={:.2f})'.format(l_t, l_t1, score))

visualize_matches(frames[t], frames[t + 1], blobs_t, blobs_t1,
                  matches_desc, title='Descriptor-based Matching')
```

Descriptor-based matches: 6

```
blob 3 -> blob 3 (score=0.00)
blob 4 -> blob 4 (score=1.15)
blob 5 -> blob 6 (score=1.39)
blob 6 -> blob 5 (score=2.18)
blob 7 -> blob 7 (score=0.86)
blob 8 -> blob 8 (score=7.01)
```



7.1 Analysis: Distance-based vs Descriptor-based matching

Distance-based — fast and intuitive. Works well when objects are well separated and their displacement between consecutive frames is smaller than `max_dist`. Fails when two blobs in $t+1$ are at similar distances from a blob in t (assignment ambiguity), or when an object moves rapidly.

Descriptor-based — augments position with area and mean intensity similarity. Reduces ambiguity when two objects of different sizes cross paths: the blob that is more similar in shape and appearance is preferred over the merely closest one. The weights `w_pos`, `w_area`, `w_intensity` allow tuning the relative importance of position vs appearance depending on the video being analysed.

8 Personalized Video

8.1 Scene Description

The custom video is sourced from [Pexels — Busy Zebra Crossing in Hong Kong](#) and shows a **busy pedestrian crossing in Hong Kong**, filmed from a **fixed elevated camera** mounted on a building above the street.

Property	Value
Resolution	1080 x 1920 px (portrait)
Frame rate	30 fps
Duration	~3 s (92 frames, first 3 s of source video)
Camera	Fixed, oblique top-down view
Scene	Yellow zebra crossing, large crowd, colourful building facades

8.1.1 Challenges

1. **Dense crowd and occlusion** — up to 50 pedestrians cross simultaneously in opposite directions. Individual blobs constantly merge into large super-blobs, making per-person tracking almost impossible.
2. **Moving shadows** — strong directional sunlight casts sharp shadows on the asphalt that move in sync with the pedestrians, producing false-positive blobs adjacent to each person.
3. **No clean background frame** — the first frames (used for background initialisation) show people standing still before the light turns green. This gives a cleaner B0 than the previous 2-second clip, but residual occlusion from stationary pedestrians remains.
4. **Complex static texture** — the alternating yellow/grey zebra stripes produce strong horizontal edges; JPEG compression artefacts or sub-pixel variation can trigger false detections along stripe boundaries.
5. **Multi-directional fast motion** — once the light turns green, pedestrians move in all four directions simultaneously, making centroid displacement between frames non-trivial to gate.

```
[22]: # Extract frames from the custom video if not already present
import os, subprocess
from pathlib import Path

video_path = Path("custom_1080_1920_30fps_first_3s.mp4")
frames_path = Path("frames")

existing = list(frames_path.glob("*.jpg")) if frames_path.exists() else []

if existing:
    print(f"Frames already present ({len(existing)} files) - skipping␣
↪extraction.")
else:
    frames_path.mkdir(parents=True, exist_ok=True)
    try:
        import cv2
        cap = cv2.VideoCapture(str(video_path))
        idx = 1
        while True:
            ret, frame = cap.read()
            if not ret:
                break
            cv2.imwrite(str(frames_path / f"frame{idx:04d}.jpg"), frame)
            idx += 1
        cap.release()
        print(f"Extracted {idx - 1} frames via OpenCV into '{frames_path}'.")
    except ImportError:
        result = subprocess.run(
            ["ffmpeg", "-i", str(video_path), "-vf", "fps=30",
             str(frames_path / "frame%04d.jpg"), "-y"],
```

```

        capture_output=True, text=True
    )
    n = len(list(frames_path.glob("*.jpg")))
    if n > 0:
        print(f"Extracted {n} frames via ffmpeg into '{frames_path}'.")
    else:
        print("ERROR: could not extract frames. Install opencv-python or
↳ffmpeg.")
        print(result.stderr[-500:])

```

Frames already present (92 files) - skipping extraction.

```

[23]: # Load frames and initialise background for the custom video
custom_dir = Path("frames")
custom_frames, custom_frame_paths = load_frames(custom_dir)
custom_background = initialize_background(video_path=custom_dir, num_frames=10)

print(f"Custom frames loaded : {len(custom_frames)}")
print(f"Frame shape           : {custom_frames[0].shape} (H x W)")
print(f"Background shape      : {custom_background.shape}")

```

```

Custom frames loaded : 92
Frame shape           : (1920, 1080) (H x W)
Background shape      : (1920, 1080)

```

```

[24]: fig, ax = plt.subplots(figsize=(5, 9))
ax.imshow(custom_background, cmap="gray",
          vmin=custom_background.min(), vmax=custom_background.max())
ax.set_title("Custom video -- Initial background B0\n(averaged over first 10
↳frames)")
ax.axis("off")
plt.tight_layout()
plt.show()

```

Custom video -- Initial background B0
(averaged over first 10 frames)



```
[25]: # V2 change detection on the custom video
custom_change_maps_v2 = compute_change_maps_with_update_backgroundv2(
    custom_frames, custom_background,
    threshold=30, alpha=0.05, motion_threshold=0.15,
)
```

```
[26]: # Otsu change detection on the custom video
custom_change_maps_otsu, custom_otsu_thresholds = compute_change_maps_otsu(
    custom_frames, custom_background
)

print("Otsu threshold -- min: {:.1f} max: {:.1f} mean: {:.1f}".format(
    min(custom_otsu_thresholds),
    max(custom_otsu_thresholds),
    sum(custom_otsu_thresholds) / len(custom_otsu_thresholds),
))
```

Otsu threshold -- min: 6.4 max: 37.4 mean: 31.0

```
[27]: # Visual comparison: V2 fixed threshold vs Otsu
indices = [0, len(custom_frames) // 2, len(custom_frames) - 1]
fig, axes = plt.subplots(3, 3, figsize=(12, 20))
for row, idx in enumerate(indices):
    axes[row, 0].imshow(custom_frames[idx], cmap="gray")
    axes[row, 0].set_title(f"Frame {idx}")
    axes[row, 0].axis("off")

    axes[row, 1].imshow(custom_change_maps_v2[idx], cmap="gray", vmin=0,
↪vmax=255)
    axes[row, 1].set_title(f"V2 tau=30 -- frame {idx}")
    axes[row, 1].axis("off")

    axes[row, 2].imshow(custom_change_maps_otsu[idx], cmap="gray", vmin=0,
↪vmax=255)
    axes[row, 2].set_title("Otsu tau={:.1f} -- frame {}".format(
        custom_otsu_thresholds[idx], idx))
    axes[row, 2].axis("off")

plt.suptitle("Custom video: original | V2 fixed tau=30 | Otsu", fontsize=12)
plt.tight_layout()
plt.show()
```

Frame 0



Custom video; original | V2 fixed tau=30 | Otsu



Otsu tau=15.2 -- frame 0



Frame 46



V2 tau=30 -- frame 46



Otsu tau=33.0 -- frame 46



Frame 91



V2 tau=30 -- frame 91



Otsu tau=37.2 -- frame 91



8.2 Analysis: V2 vs Otsu on the Custom Video

The improved background (static crowd in first 10 frames) makes frame 0 almost empty with V2 — correct, since stationary people are absorbed into B0.

Otsu at frame 0 ($\tau=15.2$) undershoots the fixed threshold: with very few moving pixels the histogram is unimodal and Otsu falls into the noise floor, over-detecting. From frame 46 onward Otsu converges to $\tau \sim 33-37$, nearly identical to V2.

Tree-shadow artefact: the lit area on the left shows moving shadows from a tree outside frame. Both methods flag them as foreground — indistinguishable from real motion since the mechanism is the same ($|I_t - B| > \tau$). A per-pixel variance model (e.g. GMM) would be needed to tolerate these low-amplitude periodic oscillations.

Verdict: the fixed threshold is preferable here — it correctly ignores the static phase while Otsu over-fires on it.

9 Blob - Custom video

```
[28]: # min_size=500 scaled for 1080x1920 (vs 20 for 640x480)
def compute_blobs_hd(change_map, min_size=300):
    passmask = morphology.disk(3)
    opened   = morphology.binary_opening(change_map, passmask)
    labeled   = measure.label(opened, connectivity=2)
    cleaned   = morphology.remove_small_objects(labeled, min_size=min_size)
    return cleaned

t_custom = len(custom_frames) // 2

cmap_t    = clean_map(custom_change_maps_v2[t_custom])
cmap_t1    = clean_map(custom_change_maps_v2[t_custom + 1])

blobs_custom_t    = compute_blobs_hd(cmap_t)
blobs_custom_t1    = compute_blobs_hd(cmap_t1)

print(f"Frame t={t_custom}:    {blobs_custom_t.max()} blobs (min_size=300)")
print(f"Frame t+1={t_custom+1}: {blobs_custom_t1.max()} blobs (min_size=300)")
```

```
Frame t=46:    327 blobs (min_size=300)
```

```
Frame t+1=47:  348 blobs (min_size=300)
```

```
[29]: # Distance-based matching
matches_dist_custom = match_blobs_distance(blobs_custom_t, blobs_custom_t1,
↪max_dist=60)
```

```

print(f"Distance-based matches: {len(matches_dist_custom)}")
for l_t, l_t1, d in matches_dist_custom:
    print(f"    blob {l_t} -> blob {l_t1}    (dist={d:.1f}px)")

visualize_matches(
    custom_frames[t_custom], custom_frames[t_custom + 1],
    blobs_custom_t, blobs_custom_t1,
    matches_dist_custom, title="Custom video - Distance-based Matching"
)

```

Distance-based matches: 86

```

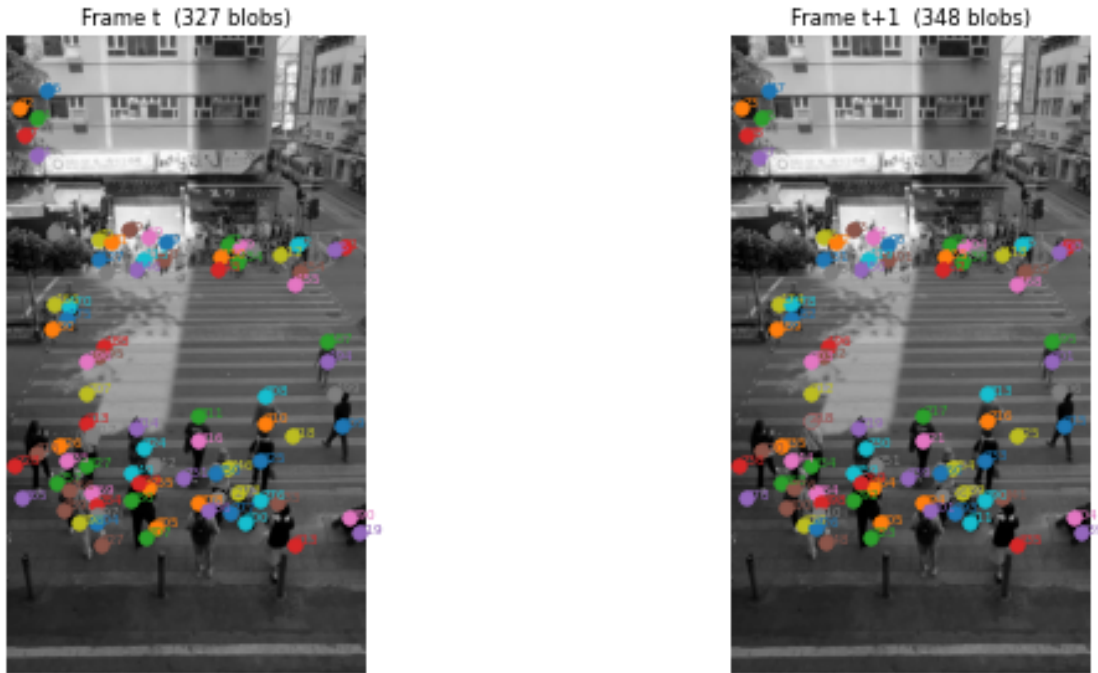
blob 16 -> blob 17    (dist=1.0px)
blob 22 -> blob 23    (dist=0.2px)
blob 24 -> blob 25    (dist=0.3px)
blob 27 -> blob 28    (dist=1.2px)
blob 34 -> blob 37    (dist=2.4px)
blob 49 -> blob 53    (dist=0.2px)
blob 50 -> blob 54    (dist=2.1px)
blob 51 -> blob 55    (dist=1.3px)
blob 70 -> blob 76    (dist=0.3px)
blob 72 -> blob 75    (dist=0.1px)
blob 79 -> blob 96    (dist=4.6px)
blob 81 -> blob 82    (dist=0.3px)
blob 83 -> blob 86    (dist=2.0px)
blob 91 -> blob 93    (dist=0.3px)
blob 95 -> blob 100   (dist=1.3px)
blob 97 -> blob 101   (dist=0.6px)
blob 99 -> blob 104   (dist=0.3px)
blob 103 -> blob 106  (dist=0.6px)
blob 112 -> blob 118  (dist=0.4px)
blob 115 -> blob 119  (dist=0.2px)
blob 127 -> blob 131  (dist=0.2px)
blob 130 -> blob 135  (dist=1.6px)
blob 134 -> blob 139  (dist=1.0px)
blob 145 -> blob 152  (dist=1.7px)
blob 146 -> blob 150  (dist=0.7px)
blob 154 -> blob 162  (dist=1.2px)
blob 155 -> blob 168  (dist=6.8px)
blob 158 -> blob 164  (dist=0.7px)
blob 166 -> blob 174  (dist=0.3px)
blob 170 -> blob 178  (dist=1.7px)
blob 175 -> blob 182  (dist=0.5px)
blob 180 -> blob 189  (dist=7.1px)
blob 187 -> blob 195  (dist=2.1px)
blob 188 -> blob 196  (dist=0.4px)
blob 194 -> blob 201  (dist=2.3px)
blob 195 -> blob 202  (dist=1.4px)
blob 196 -> blob 203  (dist=0.3px)

```

blob 199 -> blob 206 (dist=3.5px)
blob 207 -> blob 212 (dist=0.5px)
blob 208 -> blob 213 (dist=3.9px)
blob 209 -> blob 215 (dist=5.5px)
blob 210 -> blob 216 (dist=5.1px)
blob 211 -> blob 217 (dist=4.1px)
blob 213 -> blob 218 (dist=4.9px)
blob 214 -> blob 219 (dist=1.5px)
blob 215 -> blob 220 (dist=3.5px)
blob 216 -> blob 221 (dist=1.7px)
blob 217 -> blob 218 (dist=44.9px)
blob 218 -> blob 225 (dist=0.2px)
blob 224 -> blob 230 (dist=0.9px)
blob 225 -> blob 233 (dist=5.1px)
blob 226 -> blob 235 (dist=0.1px)
blob 227 -> blob 234 (dist=1.1px)
blob 228 -> blob 236 (dist=0.2px)
blob 231 -> blob 239 (dist=0.3px)
blob 233 -> blob 241 (dist=11.3px)
blob 235 -> blob 244 (dist=1.1px)
blob 242 -> blob 251 (dist=2.5px)
blob 246 -> blob 254 (dist=2.6px)
blob 249 -> blob 259 (dist=0.6px)
blob 250 -> blob 260 (dist=1.2px)
blob 255 -> blob 264 (dist=1.6px)
blob 257 -> blob 268 (dist=3.2px)
blob 259 -> blob 269 (dist=1.9px)
blob 265 -> blob 278 (dist=0.6px)
blob 266 -> blob 276 (dist=2.0px)
blob 269 -> blob 284 (dist=1.1px)
blob 272 -> blob 286 (dist=2.3px)
blob 273 -> blob 289 (dist=0.7px)
blob 276 -> blob 290 (dist=1.7px)
blob 277 -> blob 293 (dist=0.3px)
blob 278 -> blob 294 (dist=1.5px)
blob 280 -> blob 292 (dist=0.9px)
blob 284 -> blob 298 (dist=2.6px)
blob 286 -> blob 301 (dist=0.4px)
blob 289 -> blob 300 (dist=3.3px)
blob 290 -> blob 304 (dist=1.5px)
blob 297 -> blob 310 (dist=3.3px)
blob 298 -> blob 320 (dist=3.3px)
blob 300 -> blob 311 (dist=2.9px)
blob 304 -> blob 326 (dist=20.6px)
blob 305 -> blob 305 (dist=12.0px)
blob 307 -> blob 323 (dist=0.6px)
blob 313 -> blob 335 (dist=3.6px)
blob 319 -> blob 339 (dist=1.8px)

blob 327 -> blob 348 (dist=2.7px)

Custom video - Distance-based Matching



```
[30]: # Descriptor-based matching
matches_desc_custom = match_blobs_descriptor(
    blobs_custom_t, blobs_custom_t1,
    custom_frames[t_custom], custom_frames[t_custom + 1],
    max_dist=60,
)
print(f"\nDescriptor-based matches: {len(matches_desc_custom)}")
for l_t, l_t1, score in matches_desc_custom:
    print(f" blob {l_t} -> blob {l_t1} (score={score:.2f})")

visualize_matches(
    custom_frames[t_custom], custom_frames[t_custom + 1],
    blobs_custom_t, blobs_custom_t1,
    matches_desc_custom, title="Custom video - Descriptor-based Matching"
)
```

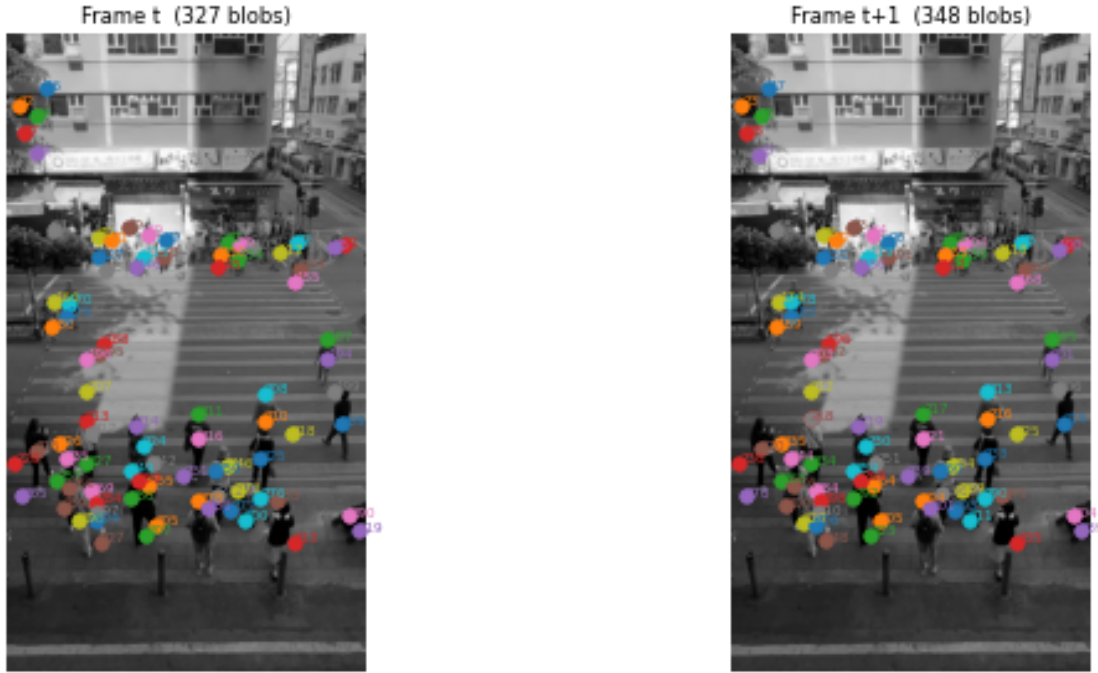
Descriptor-based matches: 86

blob 16 -> blob 17 (score=0.99)
blob 22 -> blob 23 (score=0.20)
blob 24 -> blob 25 (score=0.26)
blob 27 -> blob 28 (score=1.20)

blob 34 -> blob 37 (score=2.45)
blob 49 -> blob 53 (score=0.25)
blob 50 -> blob 54 (score=2.15)
blob 51 -> blob 55 (score=1.27)
blob 70 -> blob 76 (score=0.32)
blob 72 -> blob 75 (score=0.16)
blob 79 -> blob 96 (score=4.79)
blob 81 -> blob 82 (score=0.30)
blob 83 -> blob 86 (score=2.05)
blob 91 -> blob 93 (score=0.32)
blob 95 -> blob 100 (score=1.35)
blob 97 -> blob 101 (score=0.59)
blob 99 -> blob 104 (score=0.31)
blob 103 -> blob 106 (score=0.61)
blob 112 -> blob 118 (score=0.37)
blob 115 -> blob 119 (score=0.25)
blob 127 -> blob 131 (score=0.21)
blob 130 -> blob 135 (score=1.68)
blob 134 -> blob 139 (score=1.07)
blob 145 -> blob 152 (score=1.72)
blob 146 -> blob 150 (score=0.75)
blob 154 -> blob 162 (score=1.29)
blob 155 -> blob 168 (score=7.09)
blob 158 -> blob 164 (score=0.68)
blob 166 -> blob 174 (score=0.35)
blob 170 -> blob 178 (score=1.78)
blob 175 -> blob 182 (score=0.55)
blob 180 -> blob 189 (score=7.31)
blob 187 -> blob 195 (score=2.08)
blob 188 -> blob 196 (score=0.45)
blob 194 -> blob 201 (score=2.35)
blob 195 -> blob 202 (score=1.46)
blob 196 -> blob 203 (score=0.30)
blob 199 -> blob 206 (score=3.53)
blob 207 -> blob 212 (score=0.50)
blob 208 -> blob 213 (score=3.98)
blob 209 -> blob 215 (score=5.54)
blob 210 -> blob 216 (score=5.14)
blob 211 -> blob 217 (score=4.07)
blob 213 -> blob 218 (score=4.87)
blob 214 -> blob 219 (score=1.54)
blob 215 -> blob 220 (score=3.55)
blob 216 -> blob 221 (score=1.71)
blob 217 -> blob 218 (score=45.11)
blob 218 -> blob 225 (score=0.20)
blob 224 -> blob 230 (score=0.94)
blob 225 -> blob 233 (score=5.13)
blob 226 -> blob 235 (score=0.05)

blob 227 -> blob 234 (score=1.16)
blob 228 -> blob 236 (score=0.17)
blob 231 -> blob 239 (score=0.30)
blob 233 -> blob 241 (score=11.36)
blob 235 -> blob 244 (score=1.17)
blob 242 -> blob 251 (score=2.49)
blob 246 -> blob 254 (score=2.60)
blob 249 -> blob 259 (score=0.64)
blob 250 -> blob 260 (score=1.21)
blob 255 -> blob 264 (score=1.67)
blob 257 -> blob 268 (score=3.29)
blob 259 -> blob 269 (score=1.90)
blob 265 -> blob 278 (score=0.58)
blob 266 -> blob 276 (score=2.05)
blob 269 -> blob 284 (score=1.16)
blob 272 -> blob 286 (score=2.34)
blob 273 -> blob 289 (score=0.69)
blob 276 -> blob 290 (score=1.70)
blob 277 -> blob 293 (score=0.33)
blob 278 -> blob 294 (score=1.59)
blob 280 -> blob 292 (score=0.96)
blob 284 -> blob 298 (score=2.58)
blob 286 -> blob 301 (score=0.43)
blob 289 -> blob 300 (score=3.45)
blob 290 -> blob 304 (score=1.55)
blob 297 -> blob 310 (score=3.39)
blob 298 -> blob 320 (score=3.33)
blob 300 -> blob 311 (score=2.99)
blob 304 -> blob 326 (score=20.64)
blob 305 -> blob 305 (score=12.10)
blob 307 -> blob 323 (score=0.56)
blob 313 -> blob 335 (score=3.64)
blob 319 -> blob 339 (score=1.91)
blob 327 -> blob 348 (score=2.72)

Custom video - Descriptor-based Matching



9.1 Analysis: Blob Matching on the Custom Video

min_size sensitivity: with `min_size=300` we get ~1500 blobs — still saturated. Raising to `min_size=2000` brings the count down to 18 meaningful regions, making the matching interpretable. The right value depends on the expected object size in the scene.

One-to-many matches: the greedy algorithm has no exclusivity — multiple blobs in t can share the same nearest neighbour in $t+1$ (e.g. blob 212 and 209 both match blob 215).

max_dist gate dominates over descriptors: reducing `max_dist` from 80 to 20 px filters candidates more strictly and improves precision, but risks losing fast-moving objects (e.g. a running person can displace >20 px in $1/30$ s). With 1500 blobs all within 80 px of each other, the area and intensity terms of the descriptor score become negligible — the position term dominates and descriptor matching degenerates to distance matching. Descriptors add real value only when blobs are large and semantically distinct.

Conclusion: this scene exceeds the limits of simple background subtraction. A higher `min_size` (2000) or a morphological closing step is the minimal fix; per-person tracking would require a fundamentally different approach.

10 Conclusion

	Station video	Custom video (HK crossing)
Scene complexity	Low — sparse pedestrians	High — dense crowd
Best threshold	Otsu (adapts to illumination drift)	Fixed $\tau=30$ (Otsu undershoots at start)
Blob quality	Compact, well-separated	Merged groups, >1500 fragments
Matching	Reliable (distance or descriptor)	Unreliable without large <code>min_size</code>
Main challenge	Illumination drift	Crowd density + moving shadows

Takeaways: - Background subtraction works well on sparse, controlled scenes; it breaks down when foreground covers a large fraction of the image. - Descriptor-based matching improves over distance-based only when blobs are large and semantically distinct; with hundreds of small noisy fragments the position term dominates and the two strategies converge. - Key hyperparameters (`tau`, `min_size`, `max_dist`) are scene- and resolution-dependent and must be re-tuned for each dataset.